



Dette er en gruppeopgave. Du skal først oprette en gruppe.

Dit svar IKKE AFLEVERET

Afleveringsopgave  1 filer

Library

The First Mandatory Assignment must be developed in groups of 4 or 5 students. If there are major circumstances that prevent you from working in groups, request a dispensation from Head of Education Marc Kluge (klu@ek.dk). Please create the groups and write your names in the Excel sheet [Groups – First Mandatory Assignment.xlsx](#) in Microsoft Teams.

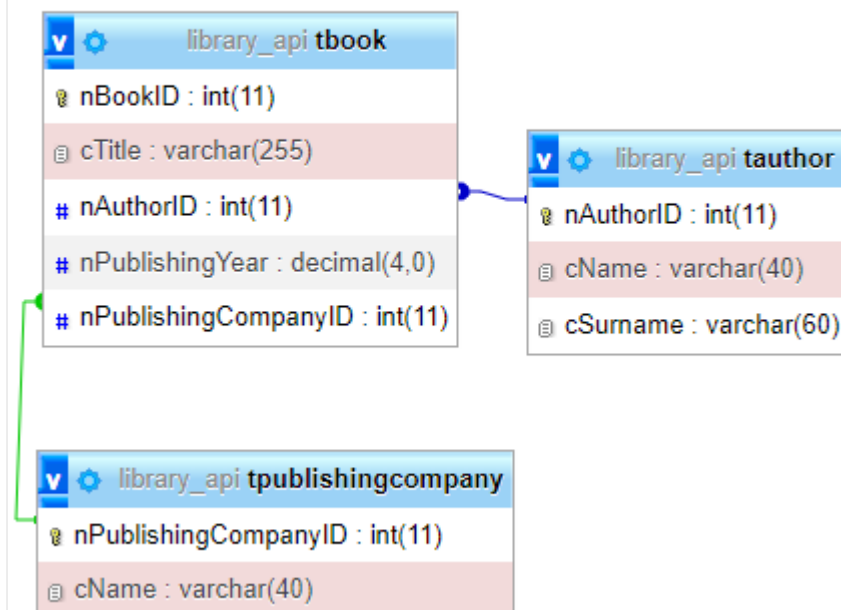
Library APIs

The assignment consists in developing four APIs for a library in the programming language(s)/library(ies)/framework(s) of your choice:

- A REST API
- A SOAP API
- A GraphQL API
- A gRPC API

Find the data source (a SQLite database) attached. Feel free to migrate it to a different DBMS. Here is its entity-relationship diagram:

Library API



REST API

Design and develop the following endpoints:

- Books: create, read by ID, list with pagination (limit and offset), update, delete
- Authors: create, read by ID, list all, update, delete
- Publishers: create, read by ID, list all, update, delete

SOAP API

Develop an HTTP endpoint for the corresponding SOAP API. Define the following XSD complex types:

- Book
 - `id (integer)`
 - `title (string)`
 - `authorId (integer)`. Must reference an existing author
 - `publishingCompanyId (integer)`. Must reference an existing publishing company
 - `publishingYear (integer)`. Minimum 1900
- Author
 - `id (integer)`
 - `name (string)`
 - `surname (string)`
- PublishingCompany
 - `id (integer)`
 - `name (string)`

Error handling will be managed via:

- `NotFoundFault`. The requested entity ID does not exist
- `ValidationFault`. Field values are missing, invalid, or referencing nonexisting entities
- `ConflictFault`. Attempt to delete an entity referenced in another entity

Operations:

- Book
 - `CreateBook`
 - **Input:** `title`, `authorId`, `publishingCompanyId`, `publishingYear`
 - **Output:** the generated ID
 - **Faults:** `ValidationFault`
 - `GetBookById`
 - **Input:** `id`
 - **Output:** `Book`
 - **Faults:** `NotFoundFault`

- UpdateBook
 - Input: id plus updated book fields
 - Output: success acknowledgement (choose your preferred format)
 - Faults:
 - ValidationFault
 - NotFoundFault
- DeleteBook
 - Input: id
 - Output: success acknowledgement (choose your preferred format)
 - Faults: NotFoundFault
- Author
 - CreateAuthor
 - Input: name, surname
 - Output: the generated ID
 - Faults: ValidationFault
 - GetAuthorById
 - Input: id
 - Output: Author
 - Faults: NotFoundFault
 - ListAuthors
 - Input: empty request
 - Output: array of Author
 - Faults: none expected
 - UpdateAuthor
 - Input: id plus updated author fields
 - Output: success acknowledgement (choose your preferred format)
 - Faults:
 - ValidationFault
 - NotFoundFault
 - DeleteAuthor
 - Input: id
 - Output: success acknowledgement (choose your preferred format)
 - Faults:
 - NotFoundFault
 - ConflictFault
- PublishingCompany
 - CreatePublishingCompany
 - Input: name
 - Output: the generated ID

- Faults: ValidationFault
- GetPublishingCompanyById
 - Input: id
 - Output: PublishingCompany
 - Faults: NotFoundFault
- ListPublishingCompanies
 - Input: empty request
 - Output: array of PublishingCompany
 - Faults: none expected
- UpdatePublishingCompany
 - Input: id plus updated publishing company fields (name)
 - Output: success acknowledgement (choose your preferred format)
 - Faults:
 - ValidationFault
 - NotFoundFault
- DeletePublishingCompany
 - Input: id
 - Output: success acknowledgement (choose your preferred format)
 - Faults:
 - NotFoundFault
 - ConflictFault

GraphQL API

Develop a POST HTTP endpoint `/graphql` for the corresponding GraphQL API. Define the following:

- Data types: Book, Author, Publisher
- Operations:
 - Query
 - Get book by ID
 - List all authors
 - List all publishers
 - Mutation
 - Create book
 - Update book by ID (all fields)
 - Delete book
 - Create author
 - Update author by ID (all fields)

- Delete author
- Create publisher
- Update publisher by ID (all fields)
- Delete publisher

gRPC API

Develop a gRPC service with the following RPCs:

- Unary: `GetBookById(GetBookByIdRequest)` returns `(Book)`
 - Parameter: book ID
- Unary: `CreateBook(CreateBookRequest)` returns `(CreateBookResponse)`
 - Parameters: book name, author ID, publisher ID, and publication year
- Streaming: `WatchBooks(WatchBooksRequest)` returns `(stream Book)`

Clients subscribed to WatchBooks will receive a message every time any other client adds a new book.

Delivery

The assignment must be delivered here in Itslearning [in one zip file](#) by 8 April 2026, 23:59:

- The source code of the four APIs
- The database script in case you migrate it to a different DBMS
- API testing media for all four APIs (e.g., Postman collections and environments)

Presentation

On **9 April 2026** each group will make a presentation where they will show and explain the whole assignment to the rest of the class:

- Demo of all four APIs via an API testing tool (e.g., Postman, Insomnia, Thunder Client)
- Walkthrough of the code
- Assessment of advantages and disadvantages of each approach for this particular scenario

The presentation will be based on delivery content. No further presentation media (e.g., Powerpoints) will be allowed.

Presentation time is limited to 15 minutes per group and all group members must present a part.